

Finishing recursive lists

5/7/25

Administrivia

- HW 7 (maze search and trees) due ~~Wednesday~~
Thursday night

Recall: Recursive methods

- Check for base case
- Otherwise, break the problem into 1 or more smaller problem instances. Solve them by calling the same method and use the results to solve the original problem.

Steps to writing a recursive list method

- Add to RecursiveList: Delegate to head
- Add to Node: Just the signature
- Add to EmptyNode: What should the method do/return on an empty list?
- Add to NonEmptyNode: What should the method do on a non-empty list? Be sure to use `next.methodName(...)`

Which of the following correctly implements get in EmptyNode?

- A. `return T;`
- B. `/* EMPTY BODY */`
- C. `throw new IndexOutOfBoundsException();`
- D. The method does not have to be implemented in this class
- E. Not exactly one of the above

Which of the following correctly implements get in EmptyNode?

- A. return T;
- B. /* EMPTY BODY */
- C. throw new IndexOutOfBoundsException();
- D. The method does not have to be implemented in this class
- E. Not exactly one of the above

T get(int index)

- In EmptyNode:
 throw new IndexOutOfBoundsException();

Which of the following correctly implements `indexOf(T val)` in `EmptyNode`?
(returns index of first occurrence; -1 if none)

- A. `/* EMPTY BODY */`
- B. `return -1;`
- C. `throw new IndexOutOfBoundsException();`
- D. `if(value.equals(val))`
 `return 0;`
 `return -1;`
- E. Not exactly one of the above

Which of the following correctly implements `indexOf(T val)` in `EmptyNode`?
(returns index of first occurrence; -1 if none)

A. `/* EMPTY BODY */`

B. `return -1;`

C. `throw new IndexOutOfBoundsException();`

D. `if(value.equals(val))`

`return 0;`

`return -1;`

E. Not exactly one of the above

Which of the following correctly implements
 `indexOf(T val)` in `NonEmptyNode`?
(returns index of first occurrence; -1 if none)

- A. `return -1;`
- B. `if(value.equals(val))`
 `return 0;`
 `return -1;`
- C. `return next.indexOf(val);`
- D. `return next.indexOf(val) + 1;`
- E. Not exactly one of the above

Which of the following correctly implements
indexOf(T val) in NonEmptyNode?
(returns index of first occurrence; -1 if none)

- A. return -1;
- B. if(value.equals(val))
 return 0;
 return -1;
- C. return next.indexOf(val);
- D. return next.indexOf(val) + 1;
- E. Not exactly one of the above

What about add?

Node methods that change the structure of the list need to return the new list

Which of the following correctly implements
add(T val) in EmptyNode?
(add to end of the list and return first node)

- A. `/* EMPTY BODY */`
- B. `return val;`
- C. `return new EmptyNode();`
- D. `return new NonEmptyNode(val, this);`
- E. Not exactly one of the above

Which of the following correctly implements
add(T val) in EmptyNode?
(add to end of the list and return first node)

- A. /* EMPTY BODY */
- B. return val;
- C. return new EmptyNode();
- D. return new NonEmptyNode(val, this);
- E. Not exactly one of the above

Which of the following correctly implements
add(T val) in NonEmptyNode?
(add to end of the list and return first node)

- A. `/* EMPTY BODY */`
- B. `value = val;`
`return this;`
- C. `next = new NonEmptyNode(val, next);`
`return this;`
- D. `return next.add(val);`
- E. Not exactly one of the above

Which of the following correctly implements
add(T val) in NonEmptyNode?
(add to end of the list and return first node)

A. `/* EMPTY BODY */`

B. `value = val;`
`return this;`

C. `next = new NonEmptyNode(val, next);`
`return this;`

D. `return next.add(val);`

E. Not exactly one of the above